

ПРАКТИЧЕСКАЯ РАБОТА

Основы алгоритмизации и программирования (C#)

Тема: Алгоритмы поиска в массиве

Время выполнения: 90 минут

Максимальный балл: 20

Цель: изучить линейный и бинарный поиск, понять разницу между ними и научиться реализовывать поиск элемента в массиве на C#

Ф.И.О. обучающегося: _____ Группа: _____

Дата: «__» _____ 20__ г. Оценка: _____

1. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1. Задача поиска

Поиск — это нахождение в массиве элемента с заданным значением (или определение, что такого элемента нет). Обычно возвращают индекс найденного элемента или -1 , если элемент не найден.

1.2. Линейный поиск (Linear Search)

Самый простой способ: просматриваем элементы по очереди с начала до конца. Как только нашли нужное значение — возвращаем его индекс. Работает на любом массиве (отсортированном и нет).

Пример: ищем число 7 в массиве {3, 1, 7, 4, 9}



Код линейного поиска:

```
static int LinearSearch(int[] a, int key)
{
    for (int i = 0; i < a.Length; i++)
    {
        if (a[i] == key)
            return i;    // нашли — возвращаем индекс
    }
    return -1;          // не нашли
}
```

1.3. Бинарный поиск (Binary Search)

Работает только на отсортированном массиве. Идея: смотрим на средний элемент. Если он равен искомому — готово. Если искомое меньше — ищем в левой половине, если больше — в правой. Каждый раз отбрасываем половину массива.

Бинарный поиск гораздо быстрее линейного на больших массивах (примерно $\log_2 n$ сравнений вместо n).

Код бинарного поиска:

```

static int BinarySearch(int[] a, int key)
{
    int left = 0;
    int right = a.Length - 1;

    while (left <= right)
    {
        int mid = (left + right) / 2;
        if (a[mid] == key)
            return mid;           // нашли
        if (a[mid] < key)
            left = mid + 1;       // ищем справа
        else
            right = mid - 1;      // ищем слева
    }
    return -1;                   // не нашли
}

```

1.4. Сравнение

- Линейный — простой, работает на любом массиве, медленный на больших данных.
- Бинарный — быстрый, но массив должен быть отсортирован.

2. ПРАКТИЧЕСКИЕ ЗАДАНИЯ

Напишите программы на C#. Время — примерно по 20–22 минуты на задание.

Задание 1. Линейный поиск (5 баллов)

Считайте размер массива n , сами элементы и число key . Найдите key линейным поиском. Если элемент найден — выведите его индекс, иначе выведите «Не найден».

Пример: 5 3 1 7 4 9 $key=7 \rightarrow 2$

Пример: 5 3 1 7 4 9 $key=5 \rightarrow$ Не найден

Место для кода:

Задание 2. Метод линейного поиска (5 баллов)

Оформите линейный поиск в виде метода `static int LinearSearch(int[] a, int key)`. Метод возвращает индекс или -1 . В Main считайте данные, вызовите метод и выведите результат.

Место для кода:

Задание 3. Бинарный поиск (5 баллов)

Считайте отсортированный по возрастанию массив и число key . Найдите key бинарным поиском. Выведите индекс или «Не найден».

Пример: 6 1 3 5 7 9 11 $key=7 \rightarrow 3$

Важно: массив уже отсортирован, сортировать не нужно.

Место для кода:

Задание 4. Подсчёт количества вхождений (5 баллов)

Считайте массив и число key . Подсчитайте, сколько раз key встречается в массиве (линейным проходом). Выведите количество.

Пример: 7 2 5 2 8 2 1 2 $key=2 \rightarrow 4$

Место для кода:

3. КРИТЕРИИ ОЦЕНИВАНИЯ

Критерий	Баллы
----------	-------

Корректный ввод массива и искомого значения	1 балл
Алгоритм поиска реализован правильно	2,5 балла
Правильный вывод результата (индекс / «Не найден» / количество)	1,5 балла
Итого за одно задание / за 4 задания	5 / 20

4. РЕКОМЕНДАЦИИ

- Линейный поиск — простой цикл for с проверкой if (a[i] == key).
- Если элемент не найден — возвращайте -1 (или выводите сообщение).
- Для бинарного поиска массив должен быть отсортирован заранее.
- В бинарном поиске $mid = (left + right) / 2$ — середина текущего отрезка.
- Проверьте оба случая: элемент есть в массиве и элемента нет.

Успешного выполнения работы!